CSE110 | PROGRAMMING LANGUAGE I — EXAM PREPARATION

Advanced Branching Practice Problems

Syllabus Scope & Rules: Solve all coding problems using **only branching statements (`if` , `else if` , `else`) and arithmetic/boolean operators**. Do not use loops, arrays, or built-in `Math` methods (such as `Math.abs` , `Math.max` , or `Math.sqrt`).

Problem 1: 4-Digit Vault PIN Cryptographic Validator (Digit Extraction via / and %)

Write a Java program that reads an integer `pin` . First check if `pin` is strictly between `1000` and `9999` ; if not, print `Invalid PIN Length` . Otherwise, extract its four individual digits d_1, d_2, d_3, d_4 from left to right using integer division (`/`) and modulo (`%`), and evaluate these security rules in exact priority order:

- **Palindrome Trap:** If the PIN reads the same forwards and backwards ($d_1 = d_4$ and $d_2 = d_3$), print `Rejected: Palindrome` .
- **Arithmetic Sequence Trap:** If the step difference between consecutive digits is constant ($d_2 - d_1 = d_3 - d_2$ and $d_3 - d_2 = d_4 - d_3$), print `Rejected: Sequential Pattern` (catches patterns like `1234` , `8642` , and `5555`).
- **Master Checksum:** If the sum of the first two digits equals the product of the last two digits ($d_1 + d_2 = d_3 \times d_4$), print `Master Vault Access Granted` .
- **Parity Balance:** Otherwise, count how many digits are even. Print `Standard Access Granted` if there are exactly two even and two odd digits, or `Rejected: Parity Mismatch` otherwise.

Sample Input	Sample Output
`7534`	`Master Vault Access Granted`
`9630`	`Rejected: Sequential Pattern`
`4815`	`Standard Access Granted`

Problem 2: Flight Timezone & 12-Hour Clock Wrap-Around Calculator (Modular Time Math)

Write a Java program that reads a flight's departure time in 24-hour format (`depH` , `depM`), flight duration (`durH` , `durM`), and destination timezone offset (`tzH` , `tzM` , where both are negative for western timezones such as `-5` hours `-30` minutes). Convert all values into total minutes from midnight (`0` to `1439`) to output the local arrival time in **12-hour AM/PM format** and the **calendar day shift**:

- **Day Shift Detection:** Compare total arrival minutes against the `0` to `1439` minute bounds of the departure day to classify arrival as `Previous Day (-1)`, `Same Day`, `Next Day (+1)`, or `Two Days Later (+2)`.
- **Negative Time Wrap:** If flying west across timezones makes total arrival minutes negative, add `1440` before extracting the local arrival hour and minute.
- **12-Hour Formatting:** Map hour `0` to `12 AM`, hours `1–11` to `AM`, hour `12` to `12 PM`, and hours `13–23` to `1–11 PM`.

Sample Input	Sample Output
`dep = 22:30, dur = 6:45, tz = +2:00`	`7:15 AM (Next Day +1)`
`dep = 01:15, dur = 3:00, tz = -6:00`	`10:15 PM (Previous Day -1)`

Problem 3: 8×8 Coordinate Threat & Geometry Analyzer (2D Distance Without Math.abs)

Write a Java program that reads two coordinates `(x1, y1)` and `(x2, y2)` on an 8 × 8 chessboard grid (values `1` to `8`). Do not use `Math.abs()`; compute the positive horizontal and vertical distances `dx` and `dy` using `if` statements to negate negative differences.

- **Validation:** If any coordinate is outside `1–8`, print `Out of Bounds`. If both coordinates are identical, print `Same Square`.
- **Color Parity:** Print whether the squares share the `Same Color` or `Opposite Colors` by checking if `(x1 + y1) % 2 == (x2 + y2) % 2`.
- **Adjacent Reach:** Print `King & Rook` if `dx + dy == 1`, or `King & Bishop` if both `dx == 1` and `dy == 1`.
- **Long-Range & Jump Reach:** For non-adjacent squares, print `Rook Only` if `dx == 0` or `dy == 0`, `Bishop Only` if `dx == dy`, `Knight Only` if `dx * dy == 2`, or `Unconnected` otherwise.

Sample Input	Sample Output
`(2, 3)` and `(4, 2)`	`Opposite Colors, Knight Only`
`(5, 5)` and `(2, 2)`	`Same Color, Bishop Only`

Problem 4: 1D Interval Topology & Overlap Analyzer (Boundary Normalization)

Write a Java program that reads four integers representing two number-line intervals `[a, b]` and `[c, d]`. Since endpoints may be entered in reverse order (such as `10` and `2`), first normalize each pair using `if` swaps so that `a <= b` and `c <= d`, then classify their exact relationship:

- **Identical:** If `a == c && b == d`, print `Identical` and the interval length `b - a`.
- **Strict Containment:** If one interval lies completely inside the other, print `Contained` along with which interval is inside which.
- **Boundary Touch:** If the end of one interval equals the start of the other (`b == c` or `d == a`), print `Touching at Point` and the shared coordinate.
- **Partial Intersection:** If they overlap partially, determine the intersection `[start, end]`, printing both the `Overlap Length` and the `Total Merged Span`.
- **Disjoint:** If they do not touch, print `Disjoint` and the exact gap distance separating them.

Sample Input	Sample Output
`[10, 2]` and `[15, 6]`	`Partial Intersection: [6, 10], Overlap = 4, Merged Span = 13`
`[1, 4]` and `[9, 6]`	`Disjoint: Gap = 2`

Problem 5: Degenerate-Safe Quadratic Root & Sign Classifier (Vieta's Logic Without Math.sqrt)

Write a Java program that takes integer coefficients `a`, `b`, and `c` for $ax^2 + bx + c = 0$ and classifies the roots **without** calling `Math.sqrt()`. Guard against division by zero when `a == 0` and use Vieta's relationships ($r_1 + r_2 = -b / a$ and $r_1 \times r_2 = c / a$) to determine the signs of distinct real roots:

- **Degenerate Cases (`a == 0`):** Print `Infinite Solutions` if `b == 0 && c == 0`, `No Solution` if `b == 0 && c != 0`, or `Single Linear Root:` followed by `-c / (double) b` if `b != 0`.
- **Complex or Repeated (`a != 0`):** Compute discriminant $D = b^2 - 4ac$. Print `Complex Conjugate Roots` if $D < 0$, or `One Repeated Real Root:` followed by `-b / (2.0 * a)` if $D == 0$.
- **Distinct Real Root Signs ($D > 0$):** If `c == 0`, one root is `0` and the other has the sign of `-b / a`.
- **Non-Zero Real Roots ($D > 0$ and `c != 0`):** If `a` and `c` have opposite signs ($a \times c < 0$), print `One Positive and One Negative Root`; otherwise, print `Two Positive Roots` if $a \times b < 0$ or `Two Negative Roots` if $a \times b > 0$.

Problem 6: Compiler-Trap Code Tracing (Negative Modulo, Dangling else, and Non-Short-Circuit & / |)

In Java, single `&` and `|` on booleans evaluate **both** sides without short-circuiting (executing any variable assignments on the right-hand side), an unbraced `else` always binds to the **nearest** preceding `if` regardless of indentation, and the sign of `%` always matches the left operand (`-13 % 5` is `-3`). Show the workings and determine the exact output of the following program:

#	Java Source Code (TrapTrace.java)
1	`public class TrapTrace {`
2	`public static void main(String[] args) {`
3	`int x = -13, y = 5, z = 4;`
4	`boolean flag = false;`
5	`int rem = x % y;`
6	`int quot = x / y;`
7	`System.out.println(rem + " " + quot);`
8	`if (rem > 0)`
9	`if (quot < 0)`
10	`z = z + 10;`
11	`else`
12	`z = z - 10;`
13	`System.out.println(z);`
14	`if (z == 4 && (x + 13 == 0 || y / 0 == 1)) {`
15	`flag = !flag;`
16	`x = -x % z;`
17	`}`
18	`if (!flag & (x = x + 3) > 2) {`
19	`System.out.println("Branch A: " + x);`
20	`} else if (x == 4 | (z = z * 2) > 10) {`
21	`System.out.println("Branch B: " + x + " " + z);`
22	`}`
23	`System.out.println("Final: " + flag + " " + z);`
24	`}`
25	`}`

Self-Check Answer Key for Problem 6 (TrapTrace)

- **Line 1 Output:** `-3 -2` (`-13 % 5` keeps the dividend's negative sign; `-13 / 5` truncates toward zero).
- **Line 2 Output:** `4` (The unbraced `else` belongs to the inner `if (quot < 0)`, so when `rem > 0` evaluates to `false`, the entire nested block is skipped).
- **Line 3 Output:** `Branch B: 4 8` (In line 14, `||` short-circuits before `y / 0`, setting `flag = true` and `x = 13 % 4 = 1`; in line 18, `!flag` is `false`, but single `&` still executes `x = x + 3` making `x = 4`; in line 20, `x == 4` is `true`, and single `|` still executes `z = z * 2` making `z = 8`).
- **Line 4 Output:** `Final: true 8`